



UML Modeling, Design Patterns, and System Design & Architecture

Advanced Programming with Java and Python

```
class System {  
    def scale():  
        return true;  
}
```

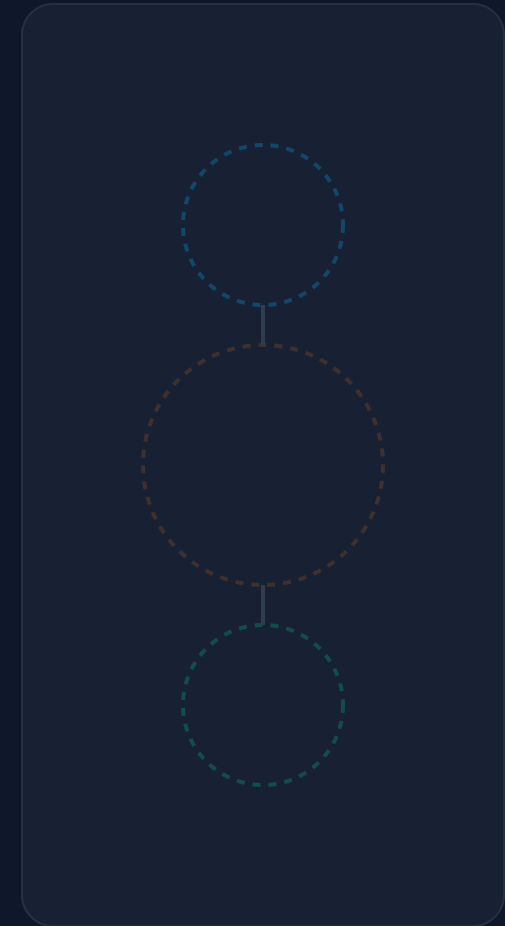
Motivation

Modern software systems must:

- ▶ Scale to millions of users
- ▶ Remain maintainable for years
- ▶ Support multiple teams
- ▶ Evolve without breaking existing features

Software engineering therefore requires:

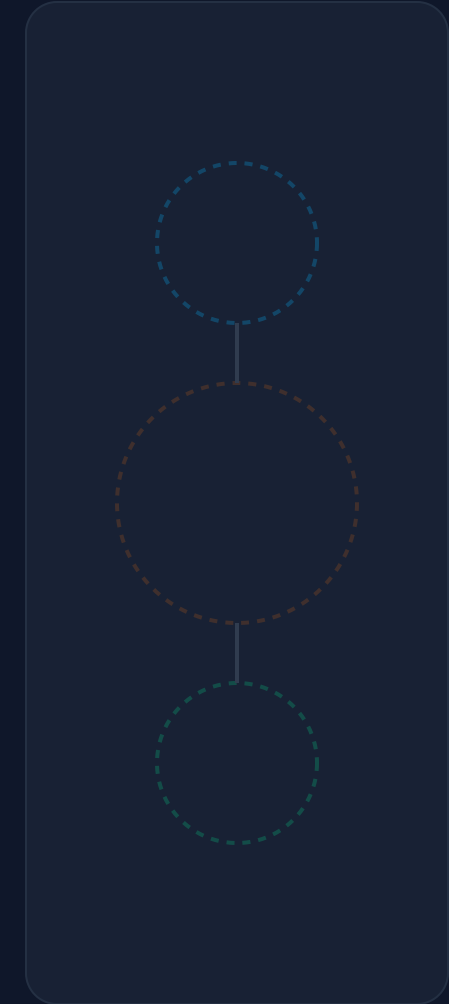
- ▶ Modeling
- ▶ Architecture planning
- ▶ Reusable design patterns
- ▶ Scalability strategies



Course Roadmap

This lecture covers:

- ▶ UML Modeling
- ▶ Design Patterns
- ▶ Microservices Architecture
- ▶ API Communication
- ▶ Database Optimization
- ▶ Caching
- ▶ Scaling
- ▶ Load Balancing
- ▶ Stateless Services
- ▶ Scalability Case Studies

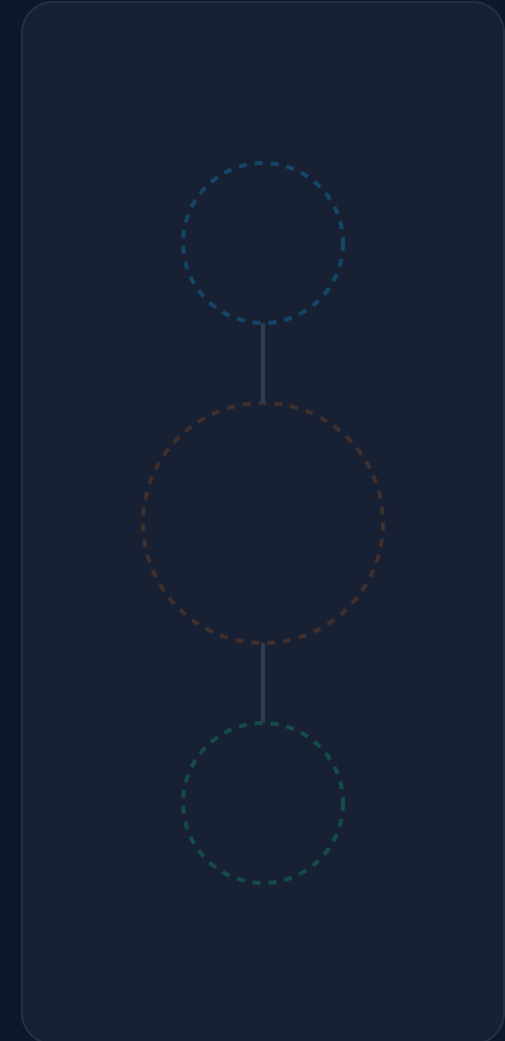


What is UML?

UML (Unified Modeling Language) is a standard visual language used to:

- ▶ Model software systems
- ▶ Visualize relationships
- ▶ Document architecture
- ▶ Communicate designs

UML is independent of programming language.



Types of UML Diagrams

■ Structural Diagrams

Class Diagram

Object Diagram

Component Diagram

■ Behavioral Diagrams

Sequence Diagram

Use Case Diagram

Activity Diagram

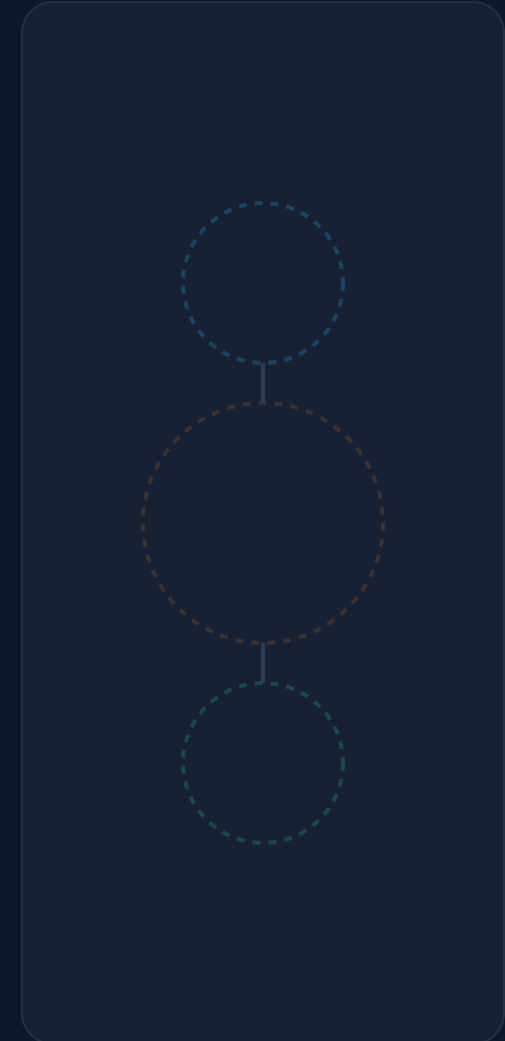
Why UML is Important

UML helps:

- ▶ Reduce ambiguity
- ▶ Simplify large systems
- ▶ Improve team communication
- ▶ Plan before coding
- ▶ Document software architecture

Industry teams use UML extensively during:

- ▶ Software analysis
- ▶ Architecture review
- ▶ Enterprise design

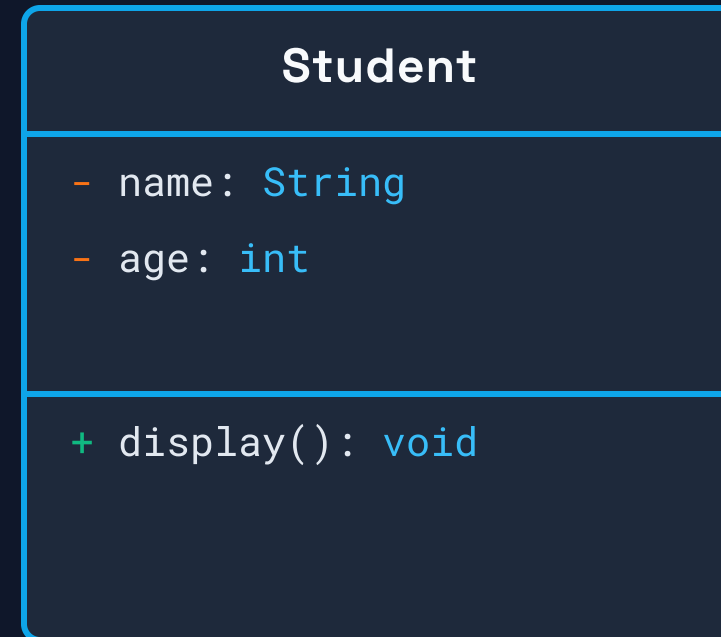


UML Class Diagram Basics

A class diagram represents:

- ▶ Classes
- ▶ Attributes
- ▶ Methods
- ▶ Relationships

Example



Java Class Example

JAVA

```
1  class Student {
2      private String name;
3      private int age;
4
5      public Student(String name, int age) {
6          this.name = name;
7          this.age = age;
8      }
9
10     public void display() {
11         System.out.println(name + " " + age);
12     }
13 }
14
```

Python Class Example

PYTHON

```
1  class Student:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6      def display(self):
7          print(self.name, self.age)
```

UML Relationships

Association

Employee knows Department

Class A — Class B

Composition

Car has Engine

◆ — Class B

Inheritance

Dog is Animal

◁ — Class B

Dependency

Service depends on Database

> - - - Class B

Composition vs Inheritance

■ Inheritance

"IS-A" relationship

Dog IS-A Animal

Promotes code reuse via hierarchy

■ Composition

"HAS-A" relationship

Car HAS-A Engine

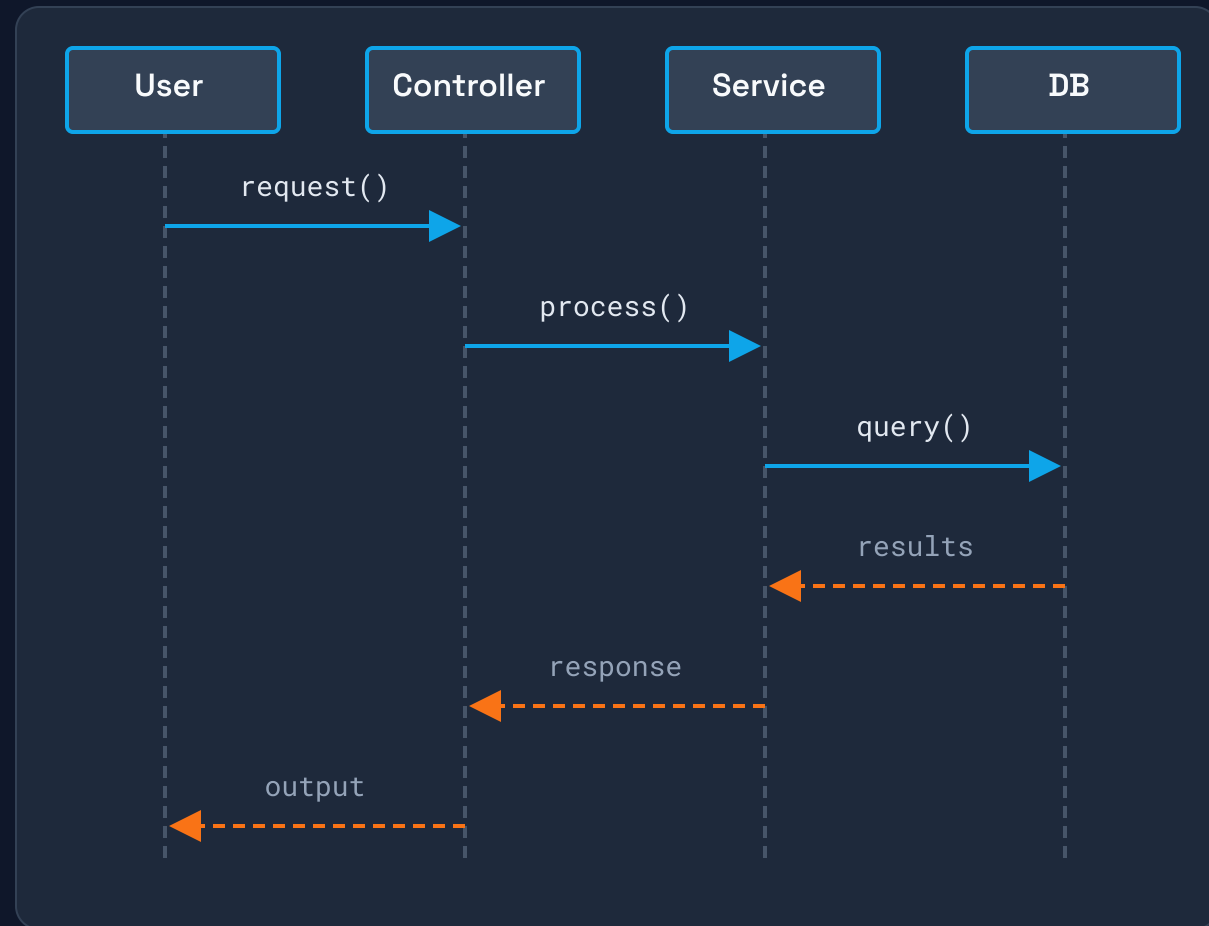
Provides loose coupling, flexibility,
better maintainability

UML Sequence Diagram

A sequence diagram shows:

- ▶ Object interactions
- ▶ Message flow
- ▶ Execution order
- ▶ Runtime behavior

i Time flows top → bottom.



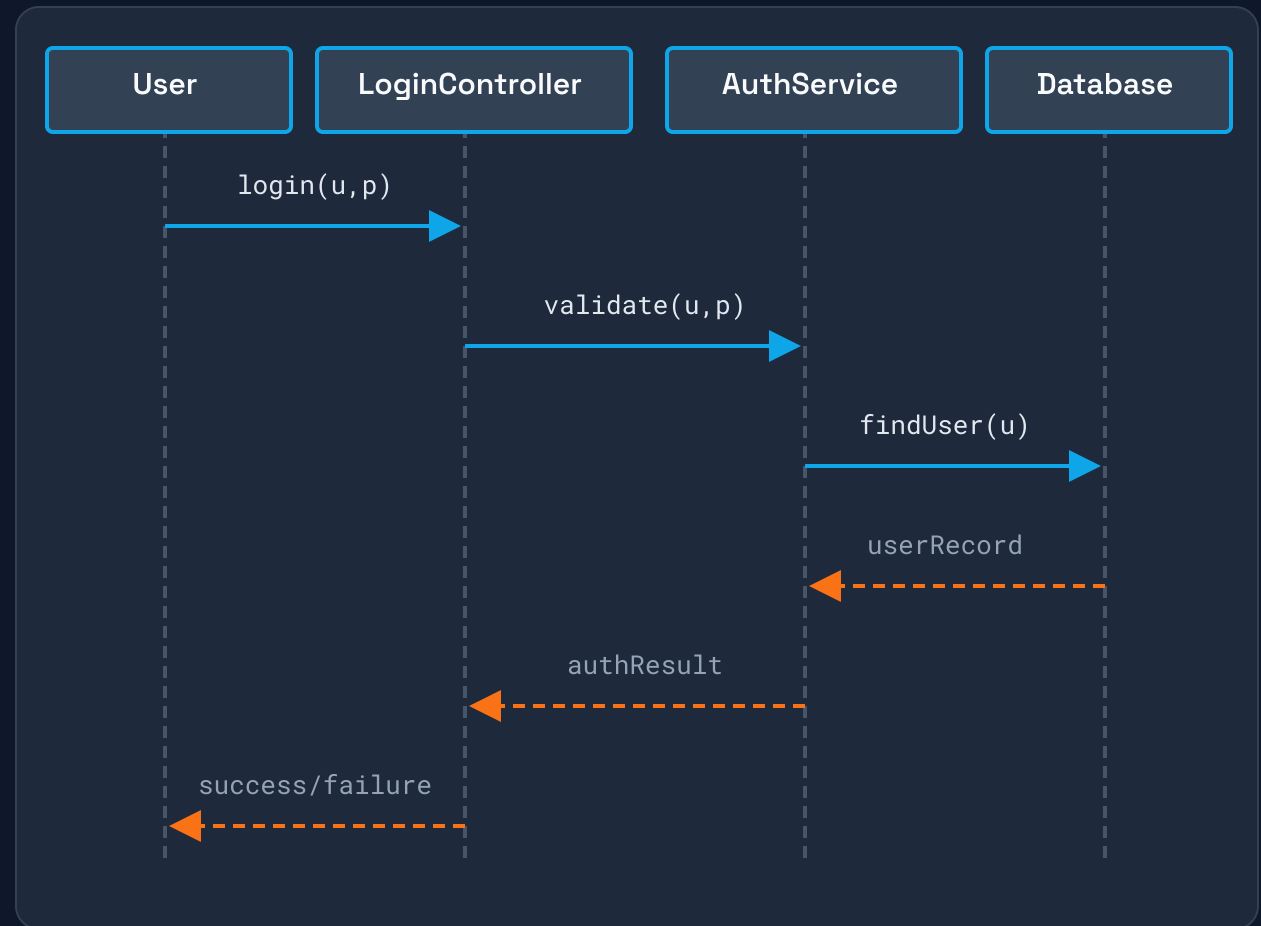
Login Sequence Diagram Case Study

Authentication Flow & Steps

► User → Controller → Service → DB

1. User submits credentials
2. Service validates credentials
3. Database verifies user
4. Result returned to user

i Security: Passwords should always be hashed.



Java Login Service

JAVA

```
1  class AuthService {  
2      public boolean login(String user, String pass) {  
3          return "admin".equals(user)  
4              && "1234".equals(pass);  
5      }  
6  }
```

Python Login Service

PYTHON

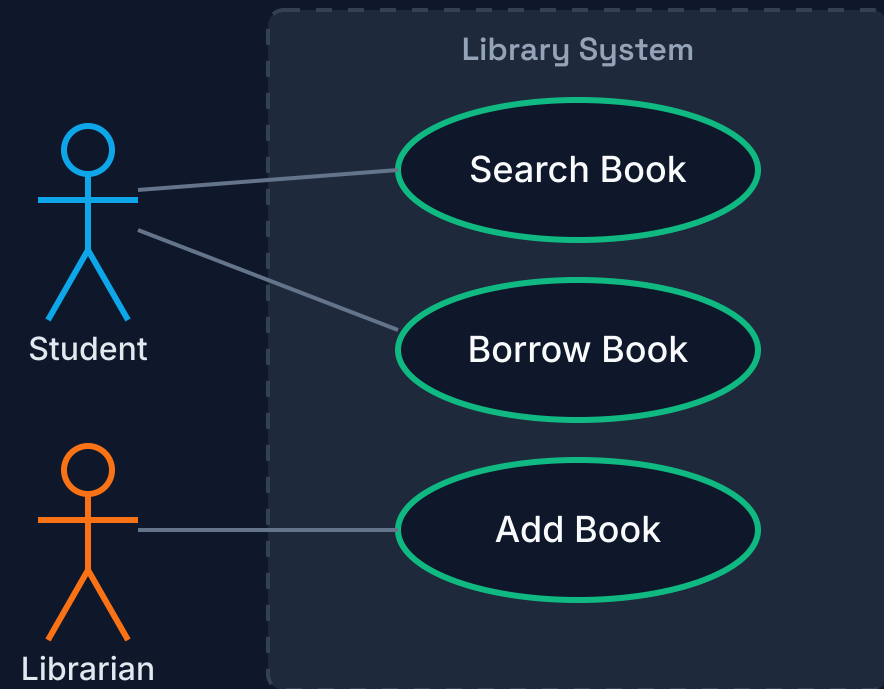
```
1 class AuthService:
2     def login(self, user, password):
3         return user == "admin" and password == "1234"
```

UML Use Case Diagram

A use case diagram models:

- ▶ System functionality
- ▶ Actors
- ▶ User interactions

Example Actors & Use Cases



Library System Use Cases

■ Student

Search Book

Borrow Book

Return Book

■ Librarian

Add Book

Remove Book

Manage Members

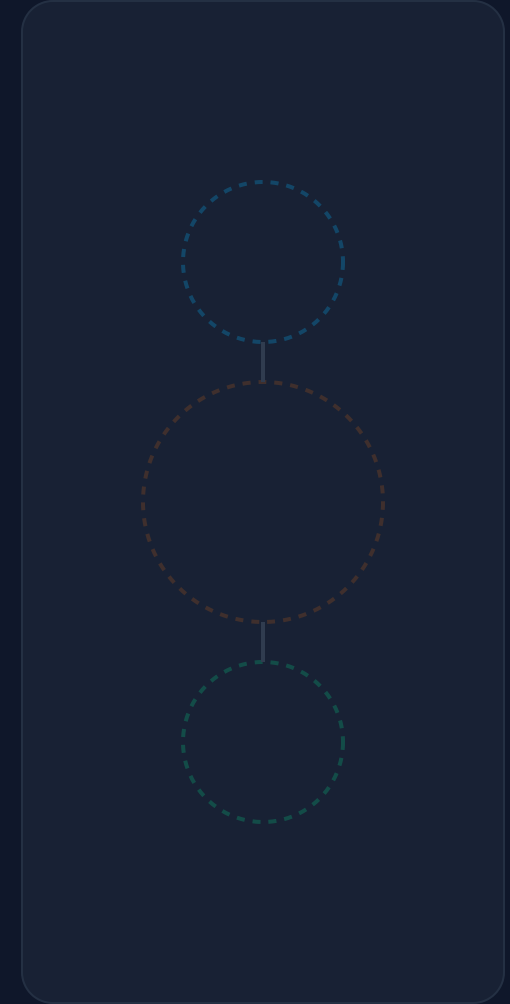
From UML to Real Software

UML helps transform:

- ▶ Requirements → design
- ▶ Design → implementation

Benefits:

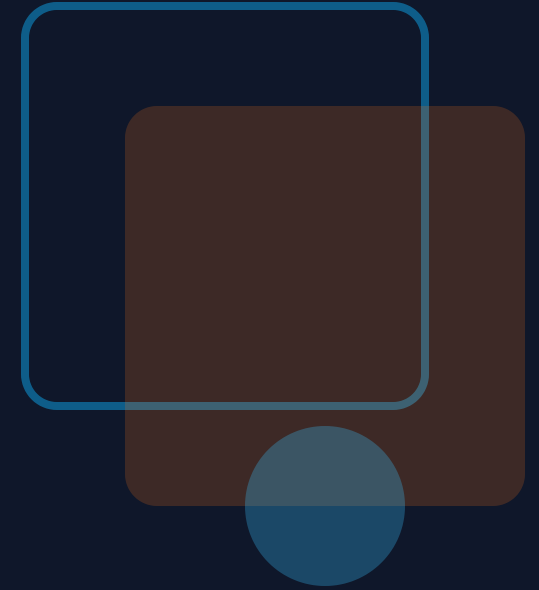
- ▶ Easier maintenance
- ▶ Better modularity
- ▶ Reduced redesign cost



02

Design Patterns

Reusable solutions to common design problems.
Use this section to frame why patterns matter before
diving into categories and examples.



Categories of Design Patterns

Creational

Object creation

⊕ new Instance()

Structural

Class/object composition

⊞ [Component A + B]

Behavioral

Object communication

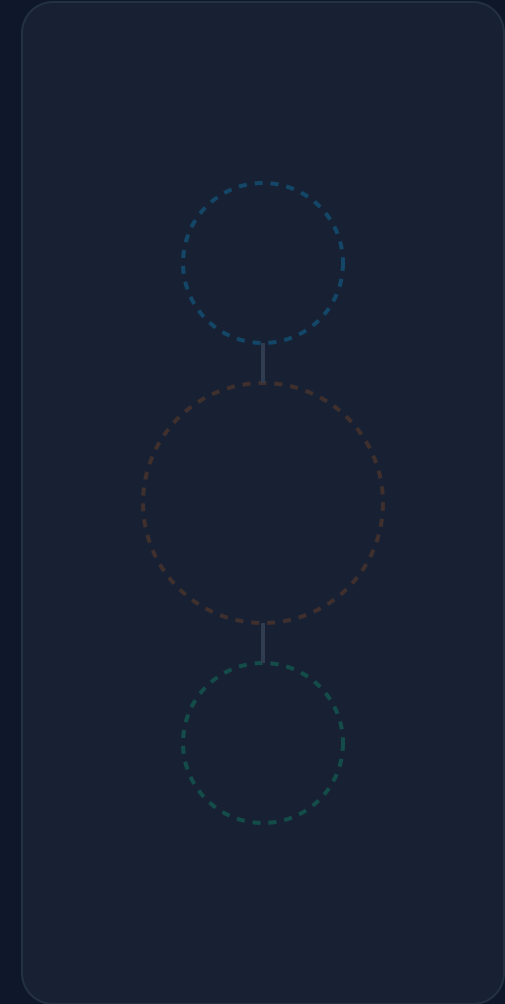
Obj A ⇔ Obj B

Creational Patterns

Important creational patterns:

- ▶ Singleton
- ▶ Factory
- ▶ Builder
- ▶ Prototype

They control object creation complexity.



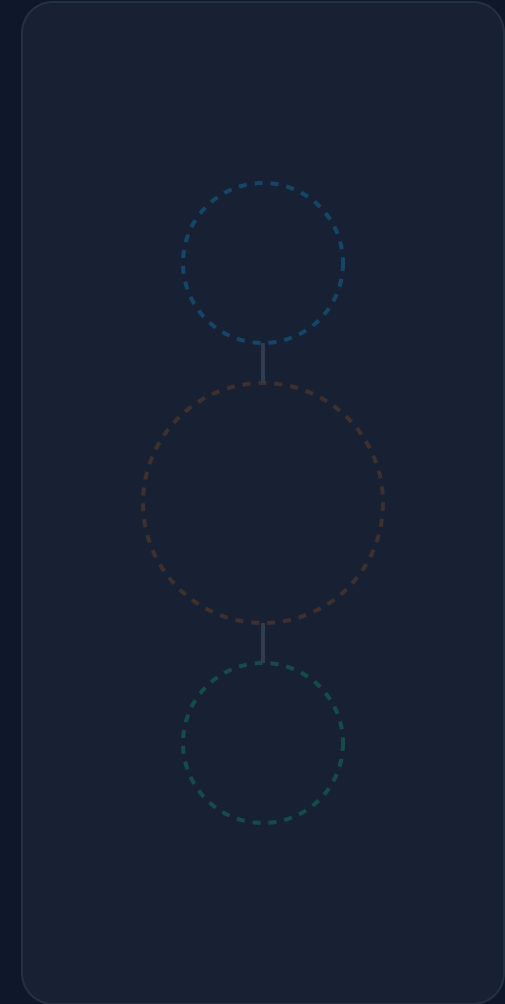
Singleton Pattern

Singleton ensures:

- ▶ Exactly one instance exists
- ▶ Global access point

Used for:

- ▶ Configuration
- ▶ Logging
- ▶ Database connection manager



Java Singleton

JAVA

```
1  class Config {  
2      private static Config instance =  
3          new Config();  
4  
5      private Config() {}  
6  
7      public static Config getInstance() {  
8          return instance;  
9      }  
10 }
```

Python Singleton

PYTHON

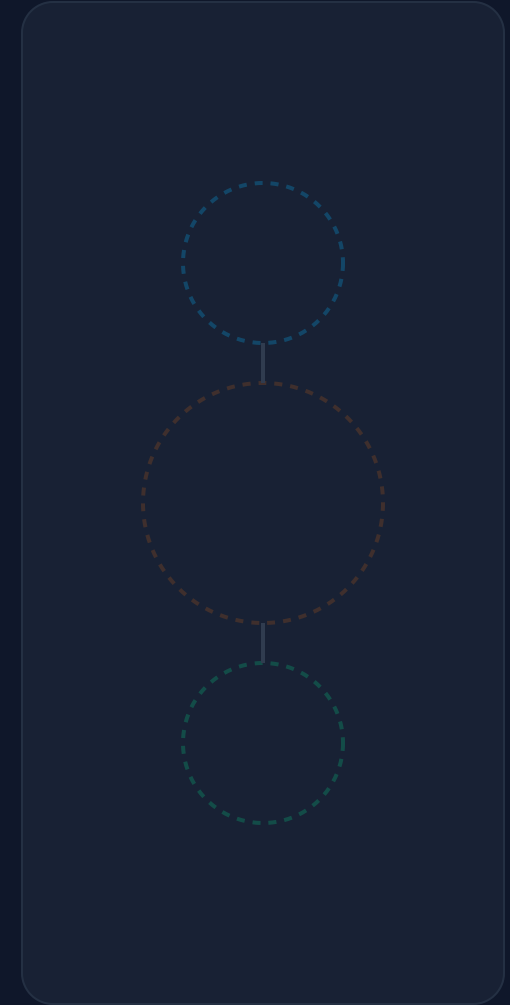
```
1  class Config:
2      _instance = None
3
4      def __new__(cls):
5          if cls._instance is None:
6              cls._instance = super().__new__(cls)
7          return cls._instance
```

Factory Pattern

Factory pattern creates objects without exposing creation logic.

Useful when:

- ▶ object type varies
- ▶ code should remain extensible



Java Factory Example

JAVA

```
1  interface Shape {  
2      void draw();  
3  }  
4  
5  class Circle implements Shape {  
6      public void draw() {  
7          System.out.println("Circle");  
8      }  
9  }
```

Python Factory Example

PYTHON

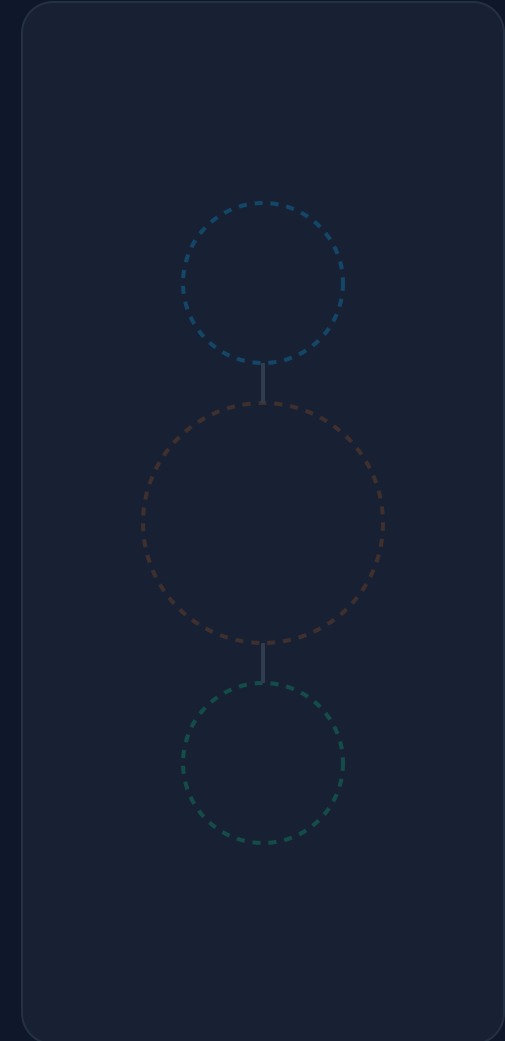
```
1  class Circle:
2      def draw(self):
3          print("Circle")
4
5  class ShapeFactory:
6      def get_shape(self, t):
7          if t == "circle":
8              return Circle()
```

Structural Patterns

Structural patterns organize classes and objects.

Important patterns:

- ▶ Adapter
- ▶ Decorator
- ▶ Facade
- ▶ Composite
- ▶ Proxy

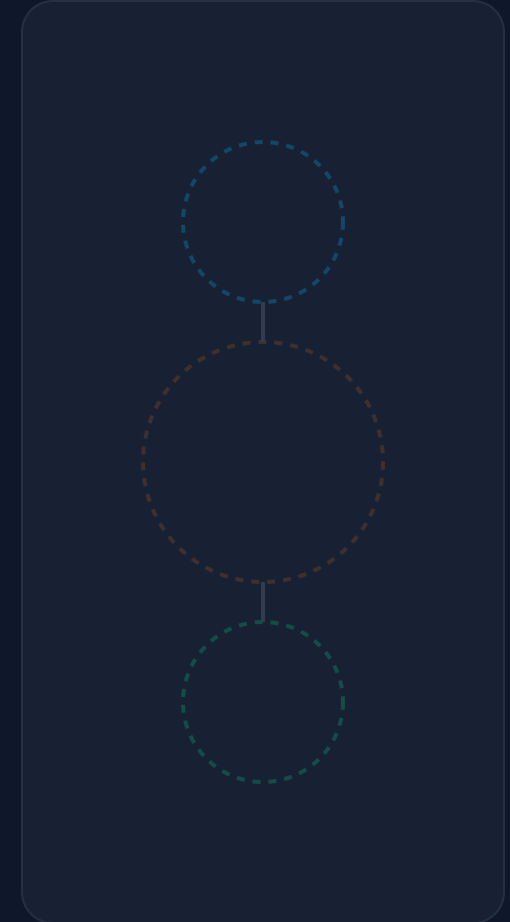


Adapter Pattern

- ▶ Adapter allows incompatible systems to communicate.

Real Example:

- ▶ Modern software integrating legacy systems.



Java Adapter Example

JAVA

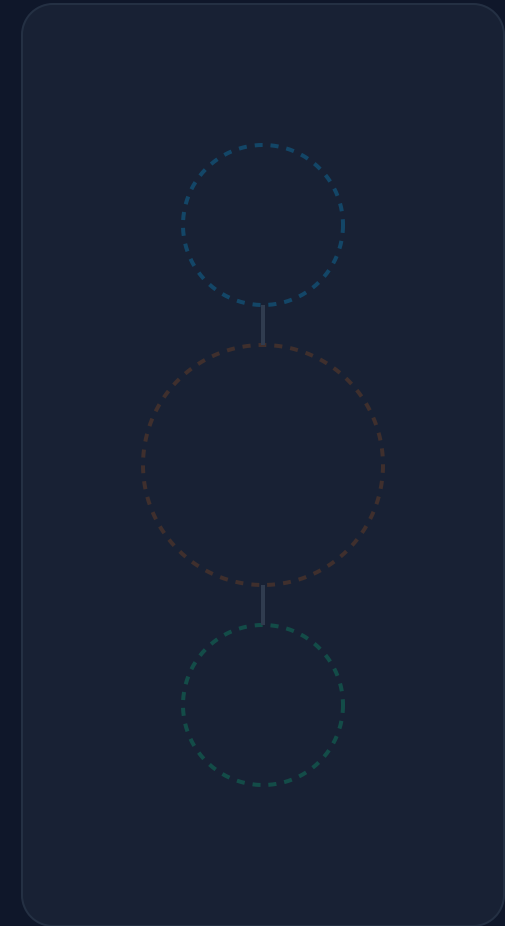
```
1  class OldPrinter {
2      void printOld() {
3          System.out.println("Old");
4      }
5  }
6
7  class Adapter {
8      private OldPrinter p = new OldPrinter();
9
10     void print() {
11         p.printOld();
12     }
13 }
```

Decorator Pattern

Decorator dynamically adds functionality.

Used for:

- ▶ logging
- ▶ security
- ▶ compression
- ▶ UI enhancements

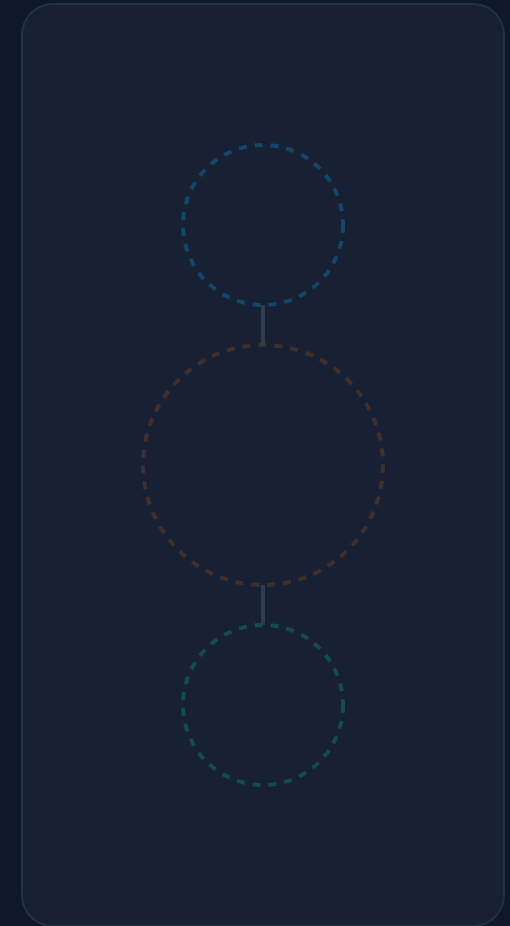


Behavioral Patterns

Behavioral patterns define communication behavior between objects and classes.

Examples:

- ▶ Strategy
- ▶ Observer
- ▶ Command
- ▶ State



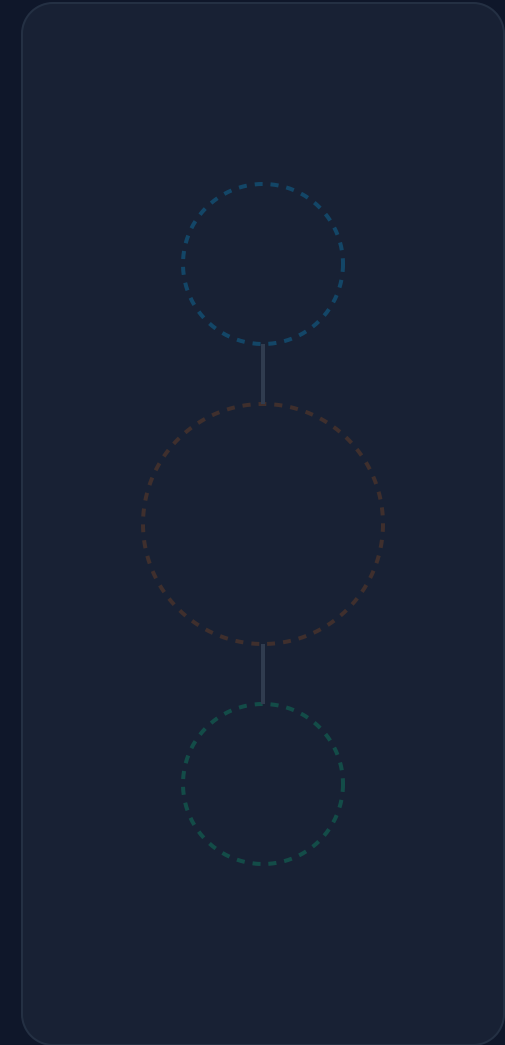
Strategy Pattern

Core Concept

- ▶ Strategy allows switching algorithms dynamically at runtime.
- ▶ Encapsulates specific behaviors behind a common interface.

Real Example: Payment methods

- ▶ UPI
- ▶ Credit Card
- ▶ Wallet



Java Strategy Example

JAVA

```
1  interface PaymentStrategy {  
2      void pay(int amount);  
3  }  
4  
5  class UPI implements PaymentStrategy {  
6      public void pay(int amount) {  
7          System.out.println("UPI");  
8      }  
9  }
```

Python Strategy Example

PYTHON

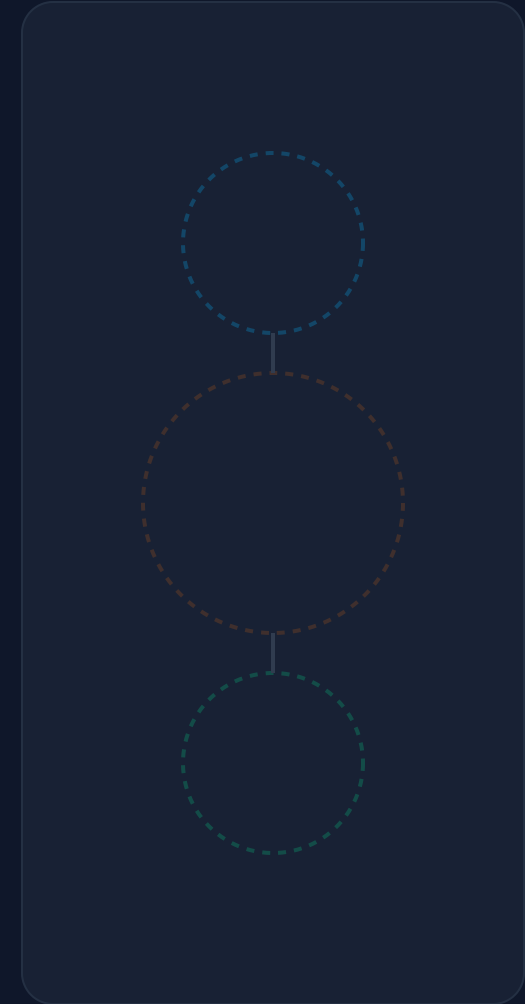
```
1 class UPI:  
2     def pay(self, amount):  
3         print("UPI Payment")
```

Observer Pattern

Observer notifies multiple subscribers automatically.

Used in:

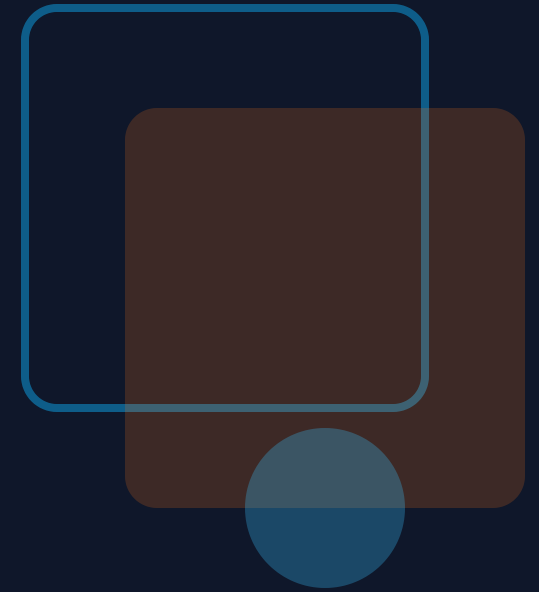
- ▶ Stock market apps
- ▶ Notifications
- ▶ Event systems



03

System Design

Focuses on scalability, reliability, availability, maintainability



Monolithic Architecture

Single deployable application.

■ Advantages

Simpler development

Easier deployment initially

■ Problems

Scaling difficulties

Tightly coupled modules

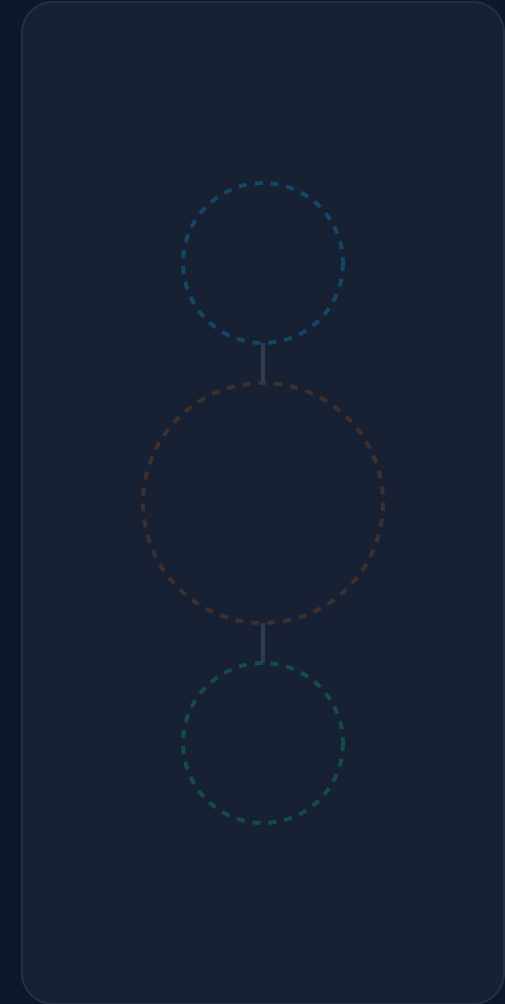
Microservices Architecture

Application divided into independent services. Examples:

- ▶ User Service
- ▶ Payment Service
- ▶ Inventory Service

Each service:

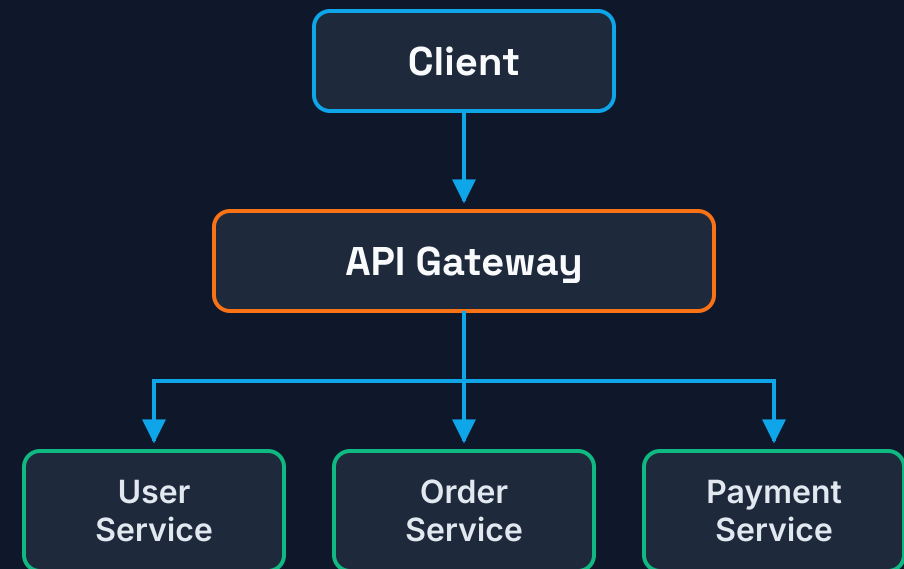
- ▶ deploys independently
- ▶ scales independently



Microservices Diagram

Component interaction:

- ▶ Client sends requests to the application
- ▶ API Gateway handles routing and traffic
- ▶ Independent microservices process logic
- ▶ Each service can scale separately



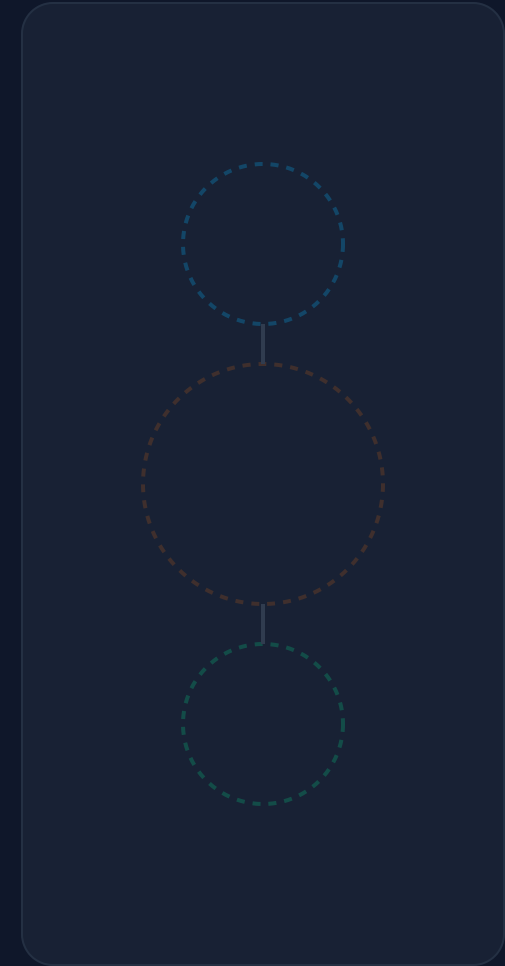
API-Based Communication

Microservices communicate using:

- ▶ REST APIs
- ▶ HTTP
- ▶ JSON
- ▶ gRPC

Benefits:

- ▶ Language independence
- ▶ Interoperability



Java REST Example

JAVA

```
1  class OrderService {  
2      public String createOrder() {  
3          return "Order Created";  
4      }  
5  }
```

Python REST Example

PYTHON

```
1 class OrderService:
2     def create_order(self):
3         return "Order Created"
```

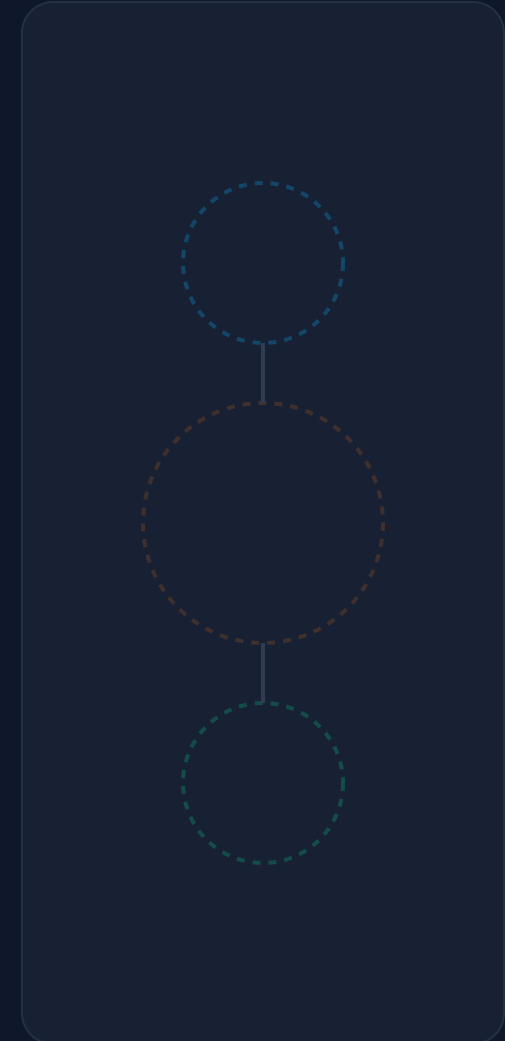
Database Optimization

Important techniques:

- ▶ Indexing
- ▶ Query optimization
- ▶ Normalization
- ▶ Denormalization

Optimization reduces:

- ▶ Latency
- ▶ CPU load
- ▶ Disk access



Indexing

■ Without Index

$O(n)$ table scan

■ With Index

$O(\log n)$ lookup

Note: Too many indexes increase storage usage and write cost.

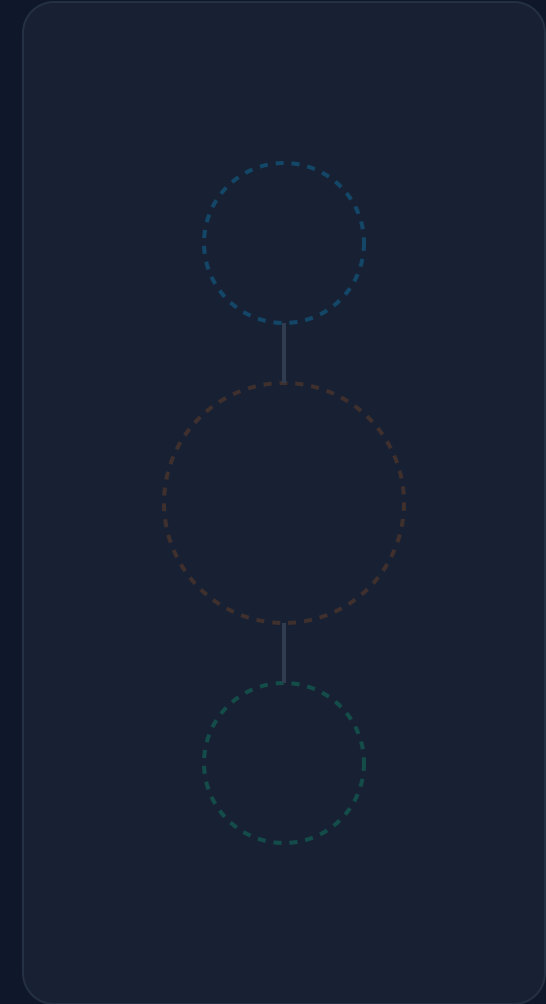
Caching Strategies

Concept & Technologies

- ▶ Stores frequently used data in memory to reduce latency
- ▶ Popular technologies: [Redis](#), [Memcached](#)

Common cache targets:

- ▶ User profiles
- ▶ Product catalogs
- ▶ Session data



Scaling Strategies

■ Vertical Scaling

Increase server power

Bigger CPU / RAM / Storage

■ Horizontal Scaling

Add more servers

Distributed architecture

Note: Large-scale systems prefer horizontal scaling.

Load Balancing

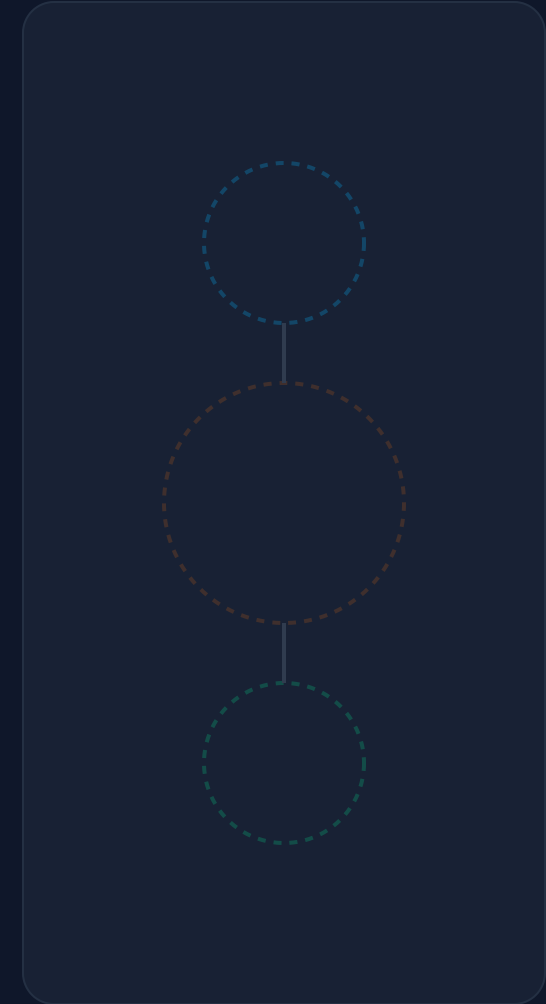
Load balancers distribute requests across servers.

Benefits:

- ▶ Fault tolerance
- ▶ Higher throughput
- ▶ Better availability

Popular algorithms:

- ▶ Round robin
- ▶ Least connections
- ▶ Weighted balancing



Stateless Services

Core Concept

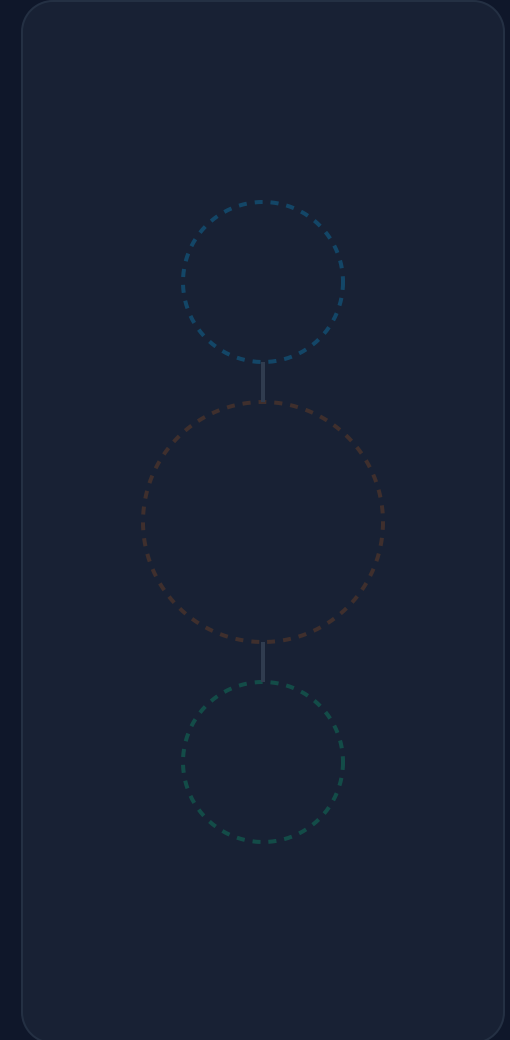
- ▶ Do not store client session locally

Advantages:

- ▶ Easier scaling
- ▶ Easier failover
- ▶ Cloud-native architecture

State stored in:

- ▶ Databases
- ▶ Distributed cache
- ▶ Tokens (e.g., JWT)



Real-World Case Study & Summary

Example: E-Commerce Platform — Uses microservices, API gateway, Redis cache, load balancing, and stateless authentication.

UML Modeling

Improves design clarity

Visualizes the system blueprint before coding

Design Patterns

Improve maintainability

Provides reusable solutions to common problems

System Architecture

Enables scalability

Supports millions of users efficiently

Good Software

Combines all three effectively

Modeling + Patterns + Architecture